

1. Lineární model báзовých funkcí

Bázové funkce a jejich příklady

- **Bázové funkce:**
 - Funkce, které nahrazují původní příznaky jejich transformovanými variantami.
 - Uvažujeme $M \in \mathbb{N}$ funkcí $\varphi_1 \dots \varphi_M$, z $\mathcal{X} \rightarrow \mathbb{R}$, kde $\mathcal{X}$ je prostor všech možných hodnot vektoru příznaku $\mathbf{X} = (X_1, \dots, X_p)^T$. Typicky $\mathcal{X} = \mathbb{R}^p$ nebo alespoň $\mathcal{X} \subset \mathbb{R}^p$.
- **Příklady:**
 - $\varphi(\mathbf{x}) = x_i$ – identita (většinou prvních p funkcí jsou identity, abychom si ponechali původní příznaky).
 - $\varphi(\mathbf{x}) = x_i^2$, $\varphi(\mathbf{x}) = x_k x_l$ – mocniny příznaků a jejich různé součiny (polynomiální regrese).
 - $\varphi(\mathbf{x}) = \log(x_i)$, $\sin(x_i)$, $\sqrt{x_i}$ atd. – nelineární transformace.
 - $\varphi(\mathbf{x}) = \mathbb{1}_{(a,b)}(x_i)$, kde $\mathbb{1}_A(x_i) = 1$, když $x_i \in A$ a $\mathbb{1}_A(x_i) = 0$, když $x_i \notin A$ --- indikátory množin. Můžu rozdělit prostor na kousky (množiny).
 - $\varphi(\mathbf{x}) = h(\|\mathbf{x} - \mathbf{x}_i\|)$, kde $\mathbf{x}_i$ je i -tý bod trénovací množiny a h je nějaká funkce --- tzv. **radiální básová funkce** centrovaná v bodech trénovací množiny.

Lineární model báзовých funkcí

- **Definice:**
 - Vysvětlovaná proměnná Y v bodě $\mathbf{x} = (x_1, \dots, x_p)^T$ hodnot příznaků $\mathbf{X}$ je určena vztahem:

$$Y = w_1 \varphi_1(\mathbf{x}) + \dots + w_M \varphi_M(\mathbf{x}) + \varepsilon = \mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) + \varepsilon,$$

kde $\boldsymbol{\varphi} : \mathcal{X} \rightarrow \mathbb{R}^M$ je vektorová funkce definovaná jako $\boldsymbol{\varphi} = (\varphi_1(\mathbf{x}), \dots, \varphi_M(\mathbf{x}))^T$ pro všechna $\mathbf{x} \in \mathcal{X}$, $\mathbf{w} = (w_1, \dots, w_M)^T$ je vektor neznámých parametrů, a ε je náhodná veličina se střední hodnotou $\mathbb{E}\varepsilon = 0$.

- Prostě vezmu normální lineární model, ale místo $\mathbf{w}^T \mathbf{x}$ mám $\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x})$.
- Model pro trénovací množinu tvořenou N dvojicemi $(Y_1, x_1), \dots, (Y_N, x_N)$ můžeme ve vektorovém tvaru zapsat jako:

$$\mathbf{Y} = \boldsymbol{\Phi} \mathbf{w} + \boldsymbol{\varepsilon},$$

kde

$$\boldsymbol{\Phi} = \begin{pmatrix} \boldsymbol{\varphi}(\mathbf{x}_1)^T \\ \vdots \\ \boldsymbol{\varphi}(\mathbf{x}_N)^T \end{pmatrix} = \begin{pmatrix} \varphi_1(\mathbf{x}_1) & \cdots & \varphi_M(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ \varphi_1(\mathbf{x}_N) & \cdots & \varphi_M(\mathbf{x}_N) \end{pmatrix}, \quad \mathbf{Y} = \begin{pmatrix} Y_1 \\ \vdots \\ Y_N \end{pmatrix}, \quad \boldsymbol{\varepsilon} = \begin{pmatrix} \varepsilon_1 \\ \vdots \\ \varepsilon_N \end{pmatrix}.$$

Explicitní řešení optimalizační úlohy trénování

- Podobně jako u standardního lineárního modelu budeme minimalizovat $\text{RSS}_\lambda(\mathbf{w})$. V našem případě ale s transformovanými příznaky, tedy:

$$\text{RSS}_\lambda(\mathbf{w}) = \|\mathbf{Y} - \Phi\mathbf{w}\|^2 + \lambda\mathbf{w}^T\mathbf{w}$$

To vede na **normální rovnici**:

$$\Phi^T\mathbf{Y} - \Phi^T\Phi\mathbf{w} - \lambda\mathbf{w} = \mathbf{0},$$

kde $\mathbf{0}$ je nulový vektor s M složkami.

Pro $\lambda > 0$ existuje jednoznačné řešení:

$$\hat{\mathbf{w}}_\lambda = (\Phi^T\Phi + \lambda\mathbf{I})^{-1}\Phi^T\mathbf{Y}.$$

Predikce hodnoty Y v bodě $\mathbf{x}$ je potom:

$$\hat{Y} = \hat{\mathbf{w}}_\lambda^T \boldsymbol{\varphi}(\mathbf{x})$$

2. Jádrová regrese

Přechod od lineárního modelu báзовých funkcí k jádrové regresi (jádrový trik)

- Duální reprezentace:**
 - Začneme s lineárním modelem báзовých funkcí:

$$\text{RSS}_\lambda(\mathbf{w}) = \|\mathbf{Y} - \Phi\mathbf{w}\|^2 + \lambda\mathbf{w}^T\mathbf{w}$$

- Ted' budeme uvažovat hodnoty $\mathbf{w}$ ve tvaru:

$$\mathbf{w} = \Phi^T\boldsymbol{\alpha} \quad \text{kde} \quad \boldsymbol{\alpha} \in \mathbb{R}^N,$$

tj. omezíme $\mathbf{w}$ na podprostor $\mathbb{R}^M$ generovaný vektory $\boldsymbol{\varphi}(\mathbf{x}_1), \dots, \boldsymbol{\varphi}(\mathbf{x}_N)$.

- Dosazením do $\text{RSS}_\lambda(\mathbf{w})$ dostáváme:

$$\text{RSS}_\lambda(\boldsymbol{\alpha}) = \|\mathbf{Y} - \Phi\Phi^T\boldsymbol{\alpha}\|^2 + \lambda\boldsymbol{\alpha}^T\Phi\Phi^T\boldsymbol{\alpha}$$

- Věta:**

Pro $\lambda > 0$ platí:

$$\min_{\boldsymbol{\alpha}} \text{RSS}_\lambda(\boldsymbol{\alpha}) = \min_{\mathbf{w}} \text{RSS}_\lambda(\mathbf{w})$$

Navíc, pokud $\mathbf{w}^*$ minimalizuje $\text{RSS}_\lambda(\mathbf{w})$, tak $\boldsymbol{\alpha}^* = \frac{1}{\lambda}(\mathbf{Y} - \Phi\mathbf{w}^*)$ minimalizuje $\text{RSS}_\lambda(\boldsymbol{\alpha})$.

Na druhou stranu, pokud $\boldsymbol{\alpha}^*$ minimalizuje $\text{RSS}_\lambda(\boldsymbol{\alpha})$, tak $\mathbf{w}^* = \Phi^T\boldsymbol{\alpha}^*$ minimalizuje $\text{RSS}_\lambda(\mathbf{w})$.

- Důkaz:**

Není potřeba ke zkoušce.

- Jádrový trik:**

- Vytvoříme ekvivalentní duální reprezentaci celého modelu (viz níže).
- V této reprezentaci se body $\mathbf{x}, \mathbf{x}_1, \dots$ vyskytují pouze ve tvaru skalárních součinů jejich transformací pomocí báзовých funkcí, které můžeme kompletně vyjádřit pomocí **jádrové funkce** $k(\mathbf{x}, \mathbf{y})$.
- Nahrazení skalárních součinů jádrovou funkcí se nazývá **jádrový trik**.
- Rovnou také můžeme začít jádrovou funkcí bez explicitního zadávání báзовých funkcí.
- Můžeme pak pracovat v **příznakových prostorech velké** (až nekonečné) **dimenze**.
- **Maticová inverze** má časovou složitost $\mathcal{O}(M^3)$ resp. $\mathcal{O}(N^3)$ pro odhad $\mathbf{w}$ resp. α .

Definice jádrové funkce a příklady

- **Definice:**

Funkci $k : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$, definovanou jako:

$$k(\mathbf{x}, \mathbf{y}) = \varphi(\mathbf{x})^T \varphi(\mathbf{y}) \quad \text{pro každé } \mathbf{x}, \mathbf{y} \in \mathbb{R}^p.$$

- **Příklady:**

- **Polynomiální jádro:**

$$k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + 1)^n$$

- **Lineární jádro:**

$$k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$$

- **Gaussovské jádro:**

$$k(\mathbf{x}, \mathbf{y}) = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2}$$

Gramova matice a explicitní řešení optimalizační úlohy trénování

- **Definice:**

Gramovu matici (angl. Gram Matrix) $\mathbf{G}$ definujeme jako:

$$\mathbf{G} = \Phi \Phi^T \in \mathbb{R}^{N, N}$$

- **Pozorování:**

- Gramova matice je **symetrická**.
 - Gramova matice je **pozitivně semidefinitní**:

$$\mathbf{a}^T \mathbf{G} \mathbf{a} = \mathbf{a}^T \Phi \Phi^T \mathbf{a} = (\Phi^T \mathbf{a})^T (\Phi^T \mathbf{a}) = \|\Phi^T \mathbf{a}\|^2 \geq 0$$

pro každé $\mathbf{a} \in \mathbb{R}^N$.

- $\text{RSS}_\lambda(\alpha)$ můžeme pomocí Gramovy matice vyjádřit jako:

$$\text{RSS}_\lambda(\boldsymbol{\alpha}) = \|\mathbf{Y} - \mathbf{G}\boldsymbol{\alpha}\|^2 + \lambda \boldsymbol{\alpha}^T \mathbf{G} \boldsymbol{\alpha}$$

- Gramova matice je **plně určena jádrovou funkcí**:

$$G_{i,j} = (\boldsymbol{\Phi}\boldsymbol{\Phi}^T)_{i,j} = \sum_{l=1}^M \Phi_{i,l} \Phi_{j,l} = \sum_{l=1}^M \varphi_l(\mathbf{x}_i) \varphi_l(\mathbf{x}_j) = k(\mathbf{x}_i, \mathbf{x}_j)$$

- **Řešení optimalizační úlohy**:

- Předpokládejme, že jsme našli $\hat{\boldsymbol{\alpha}}$ minimalizující $\text{RSS}_\lambda(\boldsymbol{\alpha})$.
- Odpovídající $\hat{\mathbf{w}} = \boldsymbol{\Phi}^T \hat{\boldsymbol{\alpha}}$ minimalizuje $\text{RSS}_\lambda(\mathbf{w})$.
- Pro predikci Y v bodě $\mathbf{x}$ máme:

$$\hat{Y} = \hat{\mathbf{w}}^T \boldsymbol{\varphi}(\mathbf{x}) = \hat{\boldsymbol{\alpha}}^T \boldsymbol{\Phi} \boldsymbol{\varphi}(\mathbf{x}) = \sum_{i=1}^N \sum_{j=1}^M \hat{\alpha}_i \Phi_{i,j} \varphi_j(\mathbf{x}) = \sum_{i=1}^N \hat{\alpha}_i k(\mathbf{x}_i, \mathbf{x})$$

- Pro gradient $\nabla \text{RSS}_\lambda(\boldsymbol{\alpha})$ (derivujeme podle $\boldsymbol{\alpha}$) platí:

$$\begin{aligned} \text{RSS}_\lambda(\boldsymbol{\alpha}) &= \|\mathbf{Y} - \mathbf{G}\boldsymbol{\alpha}\|^2 + \lambda \boldsymbol{\alpha}^T \mathbf{G} \boldsymbol{\alpha} \\ \nabla \text{RSS}_\lambda(\boldsymbol{\alpha}) &= -2\mathbf{G}(\mathbf{Y} - \mathbf{G}\boldsymbol{\alpha}) + 2\lambda \mathbf{G}\boldsymbol{\alpha}. \end{aligned}$$

- Položíme rovno nule (nulovému vektoru):

$$\mathbf{G}(\mathbf{Y} - \mathbf{G}\boldsymbol{\alpha} - \lambda \boldsymbol{\alpha}) = \mathbf{0}$$

- Protože je Gramova matice **pozitivně semidefinitní**, je $\mathbf{G} + \lambda \mathbf{I}$ regulární a dostáváme:

$$\hat{\boldsymbol{\alpha}} = (\mathbf{G} + \lambda \mathbf{I})^{-1} \mathbf{Y}$$

- Tedy i optimální $\hat{\boldsymbol{\alpha}}$ můžeme spočítat pomocí Gramovy matice a tedy jádrové funkce.

Predikce v jádrové regresi

- **Predikce** (pomocí jádrové funkce):

$$\hat{Y} = \sum_{i=1}^N \hat{\alpha}_i k(\mathbf{x}_i, \mathbf{x})$$

Jádrové modely:

- Uvažujme, že data pocházejí ze skutečného modelu:

$$Y = f(\mathbf{x}) + \varepsilon,$$

kde f je nějaká neznámá funkce a ε je náhodná veličina s $\mathbb{E}\varepsilon = 0$.

- V **lineárním modelu báзовých funkcí** hledáme f ve tvaru:

$$f(\mathbf{x}) = w_1 \varphi_1(\mathbf{x}) + \dots + w_M \varphi_M(\mathbf{x}).$$

- V obecném **jádrovém modelu** s jádrovou funkcí k máme f ve tvaru:

$$f(\mathbf{x}) = \sum_{j=1}^K \alpha_j k(\boldsymbol{\mu}_j, \mathbf{x}),$$

kde $\boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K \in \mathcal{X}$ jsou nějaké středové body.

- Speciálním případem jádrových modelů jsou **support vector machines**. Tak jim říkáme, když středové body jsou z trénovací množiny, tj:

$$f(\mathbf{x}) = \sum_{j=1}^K \alpha_j k(\mathbf{x}_j, \mathbf{x}).$$

3. Metoda podpůrných vektorů (SVM)

Geometrický popis predikce pomocí lineární diskriminační funkce

- Mějme model s lineární diskriminační funkcí:

$$f(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) + w_0,$$

kde pokud $f(\mathbf{x}) \geq 0$, predikujeme $\hat{Y}(\mathbf{x}) = 1$ a pokud $f(\mathbf{x}) < 0$, predikujeme $\hat{Y}(\mathbf{x}) = -1$.

- **Rozhodovací hranice** je nadrovina v $\mathbb{R}^M$, což je řešení této rovnice:

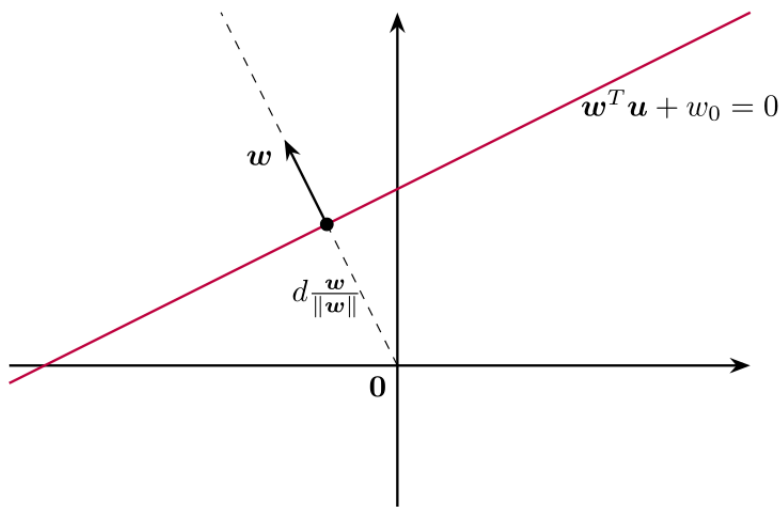
$$\mathbf{w}^T \mathbf{u} + w_0 = 0$$

- Její normála je dána vektorem $\mathbf{w}$.
- Její vzdálenost d od počátku (ve směru) $\mathbf{w}$ je určena jako:

$$\mathbf{w}^T d \frac{\mathbf{w}}{\|\mathbf{w}\|} + w_0 = d \|\mathbf{w}\| + w_0 = 0,$$

což dává

$$d = -\frac{w_0}{\|\mathbf{w}\|}.$$



Znázornění rozhodovací hranice.

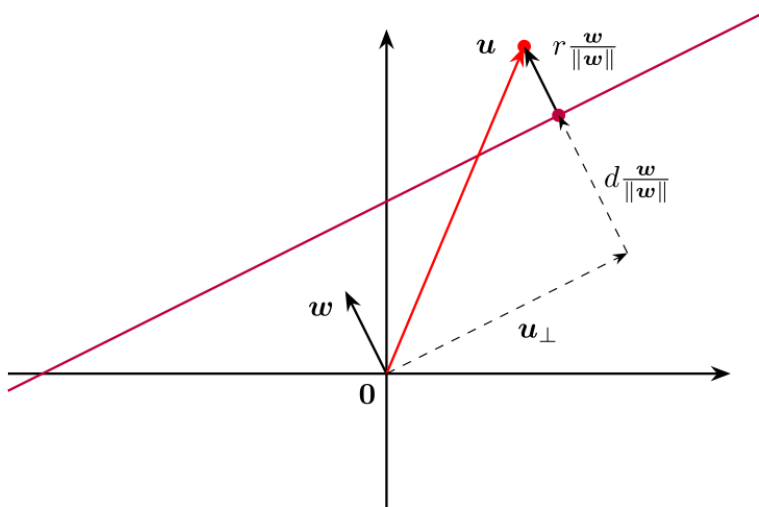
- Libovolný bod $u \in \mathbb{R}^M$ lze **jednoznačně rozložit** na část $u_{\perp}$, která je **kolmá** na w a část $u_{\parallel}$, která je **rovnoběžná** s w ,

$$u = u_{\perp} + u_{\parallel}.$$

- Dále zavedeme $u_{\parallel} = u_{\parallel} \frac{w}{\|w\|}$ a $r = u_{\parallel} - d$, dostaneme:

$$u = u_{\perp} + d \frac{w}{\|w\|} + r \frac{w}{\|w\|}$$

Modrá část $u_{\perp} + d \frac{w}{\|w\|}$ leží na rozhodovací hranici a červená část $r \frac{w}{\|w\|}$ odpovídá vzdálenosti od rozhodovací hranice ve směru w .



Vizualizace rozkladu $u = u_{\perp} + d \frac{w}{\|w\|} + r \frac{w}{\|w\|}$.

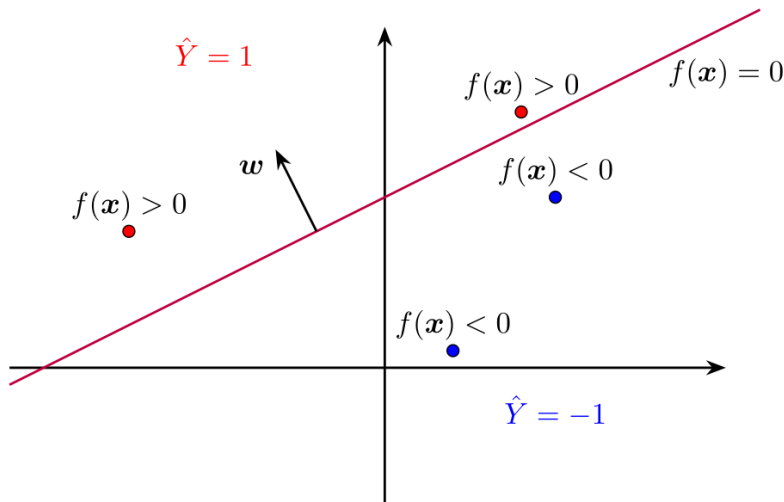
- Vzdálenost u od rozhodovací hranice je

$$r = \frac{w^T u + w_0}{\|w\|}$$

a tedy pro $u = \varphi(x)$ je vzdálenost od rozhodovací hranice

$$r = \frac{\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) + w_0}{\|\mathbf{w}\|} = \frac{f(\mathbf{x})}{\|\mathbf{w}\|}.$$

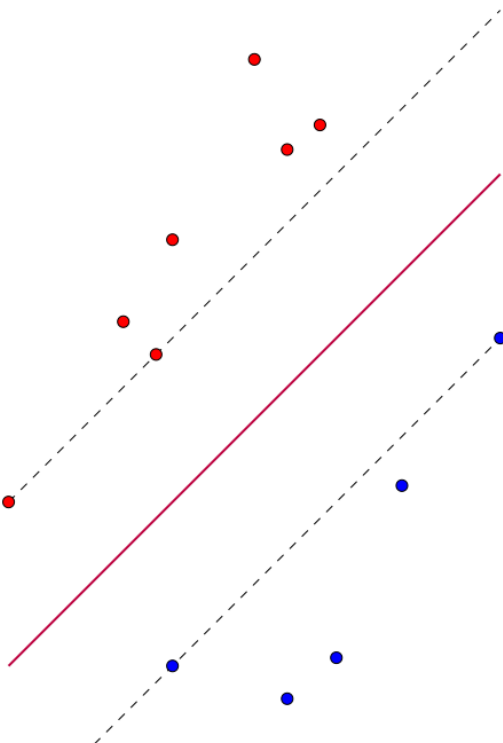
- Navíc znaménko $f(\mathbf{x})$ rozděluje prostor $\mathbb{R}^M$ na dva pod-prostory a určuje predikci ve všech bodech následovně:
 - **Pozitivní** ve směru $\mathbf{w}$: $f(\mathbf{x}) > 0 \implies \hat{Y}(\mathbf{x}) = 1$
 - **Negativní** ve směru $-\mathbf{w}$: $f(\mathbf{x}) < 0 \implies \hat{Y}(\mathbf{x}) = -1$



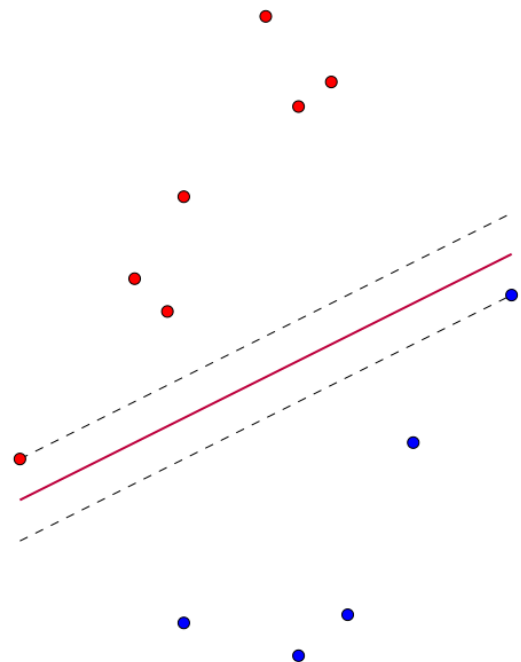
Rozhodovací hranice a predikční polo-prostory.

Metoda maximálního odstupů v lineárně separabilním případě

- Typicky existuje nekonečně mnoho řešení, které separuje dvě třídy. My vybereme takové, pro které je **vzdálenost separující nadroviny od nejbližšího bodu největší** (maximální odstup).



Separující nadrovina s větším odstupem.



Separující nadrovina s menším odstupem.

- Na obrázku vidíme, že nadrovina **vlevo** je **výhodnější pro predikci**, protože nová data, blíže separující nadrovině, mají **větší pravděpodobnost být správně zařazena**.
- Chceme pro **bod s nejmenší vzdáleností** od separující nadroviny tuto **vzdálenost největší**.
- Tedy pro vzdálenost i -tého bodu od separující nadroviny $r_i = \frac{f(\mathbf{x}_i)}{\|\mathbf{w}\|}$ chceme najít $\mathbf{w}$ a w_0 , aby minimální r_i bylo největší možné, tedy:

$$\max_{\mathbf{w}, w_0} \min_i \frac{Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0)}{\|\mathbf{w}\|}$$

a navíc musí platit, aby byl každý bod na správné straně rozhodovací hranice:

$$Y_i(f(\mathbf{x}_i)) > 0.$$

- Při přeškálování parametrů $\mathbf{w} \rightarrow \kappa \mathbf{w}$ a $w_0 \rightarrow \kappa w_0$ se vzdálenost od hranice nezmění.
- Zvolíme tedy škálu takovou, aby

$$Y_j(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) = 1$$

pro bod **nejblíže** rozhodovací hranici.

- Pro vyhovující $\mathbf{w}$ to znamená, že:

$$\min_i \frac{Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0)}{\|\mathbf{w}\|} = \frac{1}{\|\mathbf{w}\|}$$

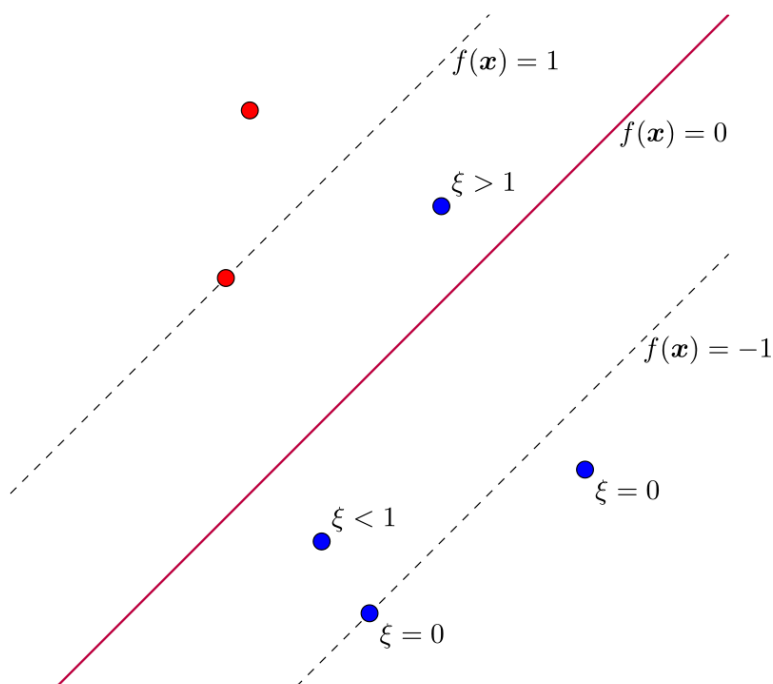
- Maximalizujeme tedy $\|\mathbf{w}\|^{-1}$ což je ekvivalentní jako minimalizování $\|\mathbf{w}\|^2$.
- Finální formulace je potom:

$$\min_{\mathbf{w}, w_0} \frac{1}{2} \|\mathbf{w}\|^2 \quad \text{za podmínky} \quad Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1 \quad \text{pro každé } i.$$

- *Toto vede na problém **kvadratického programování**.*
- Body řešení, pro které platí $Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) = 1$ se nazývají **podpůrné vektory**
- Metoda se tedy nazývá **metoda podpůrných vektorů** (angl. **support vector machine**).

Obecný lineárně neseparabilní případ

- Když data **nejsou lineárně separabilní**, nejsme schopni udržet vazbu $Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1$ pro všechny i .
- Rozvolníme vazby, aby aspoň některé body mohly tuto hranici překročit.
- Zavedeme **uvolněné proměnné (slack variables)** ξ_i následovně:
 - $\xi_i = 0$ – pokud bod i je na nebo vně pásu odstupů na správné straně.
 - $0 < \xi_i \leq 1$ – pokud bod i je uvnitř pásu odstupů, ale na správné straně.
 - $\xi_i > 1$ – pokud bod i je na špatné straně rozhodovací hranice.



Vztah uvolněných proměnných k pásmu odstupu.

- Pomocí uvolněných proměnných můžeme rozvolnit podmínky pomocí **měkkých podmínek**:

$$Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1 - \xi_i \quad \text{pro každé } i = 1, \dots, N.$$

- Přidáme ještě penalizační člen a dostáváme novou **optimalizační úlohu**:

$$\min_{\mathbf{w}, w_0, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^N \xi_i$$

vzhledem k

$$\xi_i \geq 0 \quad \text{a} \quad Y_i(\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}_i) + w_0) \geq 1 - \xi_i \quad \text{pro všechna } i.$$

- Něco něco **Lagrangeovy multiplikátory a princip duality...**
- Řešení se nazývá metoda podpůrných vektorů** (angl. **support vector machine**) (SVM)

Tvar výsledného jádrového modelu

- Řešením je **jádrový model**:

$$f(\mathbf{x}) = \sum_{j=1}^N \alpha_j k(\mathbf{x}, \mathbf{x}_j) + w_0$$

s $\alpha_j = a_j Y_j$.

- Čím **větší** parametr C , tím **méně** podpůrných vektorů dostaneme.

4. Redukce dimenzionality pomocí ortogonální projekce

Problematika vysokých dimenzí

- **Prokletí dimenzionality:**

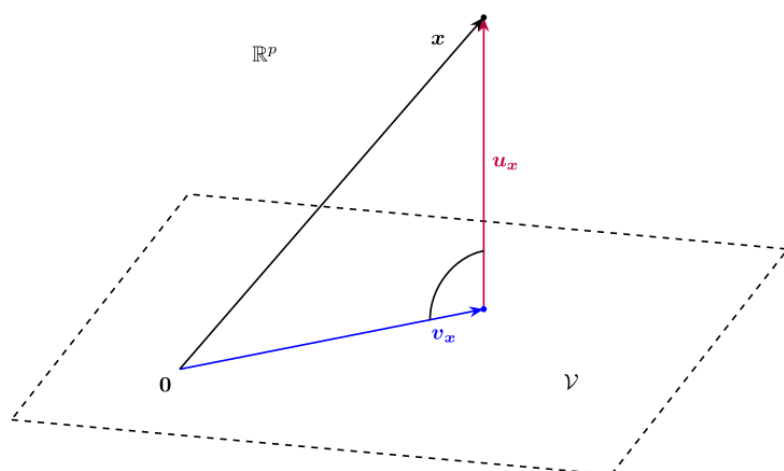
- V jednotkové krychli v d rozměrném prostoru mějme podkrychli s hranou délky $(1 - \varepsilon)$, v ní máme $(1 - \varepsilon)^d \cdot 100\%$ všech bodů. (*Předpokládáme rovnoměrné rozdělení.*)
- pro $\varepsilon = 0.1$:
 - $d = 1$: $(1 - 0.1)^1 \cdot 100\% = 90\%$ všech dat.
 - $d = 2$: $(1 - 0.1)^2 \cdot 100\% = 81\%$ všech dat.
 - $d = 10$: $(1 - 0.1)^{10} \cdot 100\% = 34.9\%$ všech dat.
 - $d = 100$: $(1 - 0.1)^{100} \cdot 100\% = 0.0027\%$ všech dat.
- S rostoucí dimenzí **exponenciálně klesá hustota** bodů, tím pádem jsou body od sebe **hodně vzdáleny**.
- Predikce jsou pak **méně spolehlivé** a model musí dělat **velké extrapolace**.
- Větší množství dat by to *teoreticky* vyřešilo, ale zase potřebujeme exponenciální zvětšování počtu dat s rostoucí dimenzí.
- Ve většině reálných případů se ale data nevyskytují rovnoměrně v prostoru, ale **vyskytují se v omezených oblastech prostoru menší dimenze**.
- Tomuto předpokladu říkáme **hypotéza variet** (angl. **manifold hypothesis**).
- Z této hypotézy dává smysl se pokoušet o **zobrazení dat do prostoru menší dimenze**.
- Rozdělíme metody na dva typy:
 - **Lineární projekce**
 - **Nelineární projekce**

Ortogonální projekce na q rozměrný podprostor

- **Lineární projekce**
- Budeme se snažit **projektovat data** z původního **příznakového prostoru** $\mathbb{R}^p$ do q **dimenzionálního podprostoru** $\mathcal{V}$.
- Víme, že každý bod $\mathbf{x} \in \mathbb{R}^p$ se dá rozložit jako:

$$\mathbf{x} = \mathbf{v}_x + \mathbf{u}_x,$$

kde $\mathbf{v}_x$ je bod vektorového prostoru $\mathcal{V}$ a $\mathbf{u}_x$ je vektor kolmý na $\mathcal{V}$.



Bod $\mathbf{v}_x$ se nazývá **ortogonální projekce** bodu $\mathbf{x}$ na podprostor $\mathcal{V}$.

- **Ortogonální projekce pomocí ortonormální báze.**

- Uvažujme **ortonormální bázi** prostoru $\mathbb{R}^p$ ($\mathbf{b}_1, \dots, \mathbf{b}_p$).
- Každý bod $\mathbf{x} \in \mathbb{R}^p$ lze napsat jako:

$$\mathbf{x} = \tau_1 \mathbf{b}_1 + \dots + \tau_q \mathbf{b}_q + \tau_{q+1} \mathbf{b}_{q+1} + \dots + \tau_p \mathbf{b}_p$$

kde i -tý koeficient $\tau_i = \mathbf{x}^T \mathbf{b}_i$.

- Pro ortogonální rozklad $\mathbf{x} = \mathbf{v}_x + \mathbf{u}_x$ tedy dostáváme:

$$\mathbf{v}_x = \tau_1 \mathbf{b}_1 + \dots + \tau_q \mathbf{b}_q \quad \text{a} \quad \mathbf{u}_x = \tau_{q+1} \mathbf{b}_{q+1} + \dots + \tau_p \mathbf{b}_p$$

- Projekci na podprostor $\mathcal{V}$ reprezentujeme jako:

$$\mathbf{t}_x = \mathbf{V}^T \mathbf{x},$$

kde $\mathbf{V} \in \mathbb{R}^{p,q}$, která ve sloupcích napsané vektory ($\mathbf{b}_1, \dots, \mathbf{b}_q$) ortonormální báze $\mathcal{V}$.
Tedy:

$$\mathbf{V} = \begin{pmatrix} | & | & & | \\ \mathbf{b}_1 & \mathbf{b}_2 & \dots & \mathbf{b}_q \\ | & | & & | \end{pmatrix}$$

Projekce jednoho bodu i celého datasetu

- **Projekce jednoho bodu:**

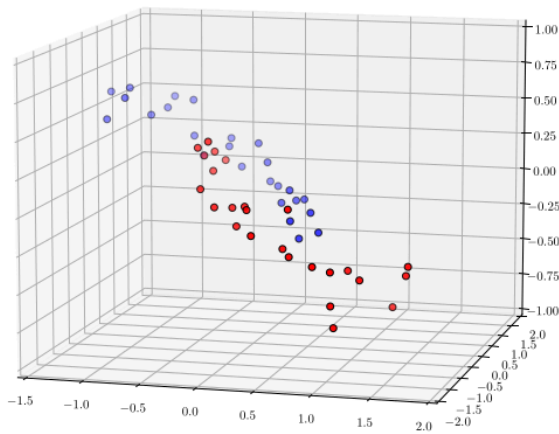
- Projekci jednoho bodu $\mathbf{x} \in \mathbb{R}^p$ realizujeme jako:

$$\mathbf{v}_x = \mathbf{V} \mathbf{t}_x = \mathbf{V} \mathbf{V}^T \mathbf{x} \quad \text{nebo} \quad \mathbf{v}_x^T = \mathbf{x}^T \mathbf{V} \mathbf{V}^T$$

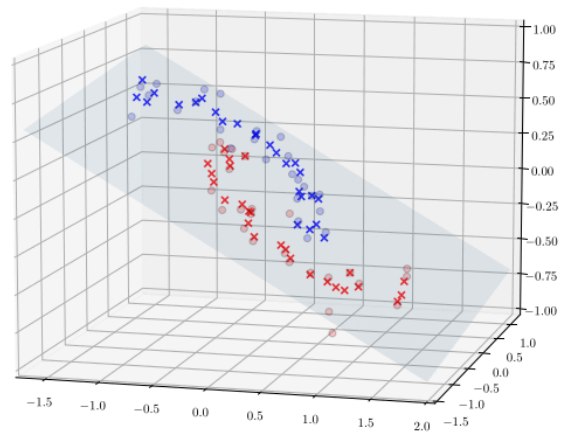
- **Projekce celého datasetu:**

- Projekci celého datasetu $\mathbf{X}_{\mathcal{V}}$ realizujeme jako:

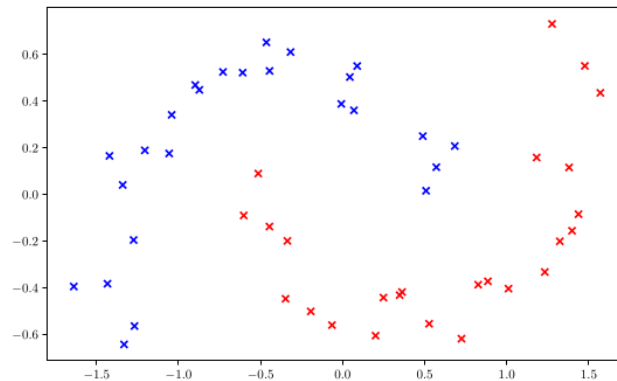
$$\mathbf{X}_{\mathcal{V}} = \mathbf{X} \mathbf{V} \mathbf{V}^T, \quad \mathbf{X}_{\mathcal{V}} \in \mathbb{R}^{N,q}$$



Originální dataset $\mathbf{X}$



Projekce $\mathbf{X}_{\mathcal{V}}$ datasetu na podprostor $\mathcal{V}$



Transformovaný dataset $\mathbf{T}_q$

- **Pozorování:**

$$\|\mathbf{v}_x\|^2 = \|\mathbf{V}\mathbf{V}^T \mathbf{x}\|^2 = \mathbf{x}^T \mathbf{V}\mathbf{V}^T \mathbf{V}\mathbf{V}^T \mathbf{x} = \mathbf{x}^T \mathbf{V}\mathbf{V}^T \mathbf{x} = \mathbf{t}_x^T \mathbf{t}_x = \|\mathbf{t}_x\|^2$$

Rovnost norem znamená, že **transformace do menšího prostoru nemění vzdálenosti od počátku**. Tedy nedochází k deformaci dat.

5. Analýza hlavních komponent (PCA)

- Jak najít pro **každé** q příslušný podprostor $\mathcal{V}$?

Optimalizační kritérium v metodě hlavních komponent

- Chceme, aby pro každé q byla **minimalizovaná kvadratická chyba projekce**.
- To jde ruku v ruce s tím, že najdeme takový podprostor, pro který budou mít transformovaná data **největší rozptyl**.

Postup provedení metody hlavních komponent

- **Vystředujeme dataset:**

$$\mathbf{x}'_i = \mathbf{x}_i - \bar{\mathbf{x}}, \quad \text{kde } \bar{\mathbf{x}} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i$$

$$\mathbf{X}' = \begin{pmatrix} - & \mathbf{x}'_1 & - \\ - & \mathbf{x}'_2 & - \\ & \vdots & \\ - & \mathbf{x}'_N & - \end{pmatrix}$$

- **Spočítáme varianční matici:**

$$\Sigma = \frac{1}{N-1} \mathbf{X}'^T \mathbf{X}', \quad \Sigma \in \mathbb{R}^{p,p}$$

- **Uděláme spektrální rozklad této matice:**

- Dostaneme vlastní čísla, která ještě **seřadíme sestupně** (od největšího po nejmenší).
- Máme tedy **vlastní čísla** ($\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$) (pokud má vlastní číslo λ_i násobnost $m > 1$, uvažujeme ho m krát)
- K nim dostaneme příslušné **vlastní vektory** ($\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_p$) s normou $\|\mathbf{b}_i\| = 1$ pro všechna $i = 1, \dots, p$.

- **Vytvoříme podprostor $\mathcal{V}$:**

- Pro podprostor $\mathcal{V}$ dimenze q vybereme **prvních q vlastních vektorů** ($\mathbf{b}_1, \dots, \mathbf{b}_q$) jako bázi tohoto podprostoru.
- Příslušná matice $\mathbf{V}$, která bude sloužit k transformaci do menší dimenze, pak vypadá následovně:

$$\mathbf{V} = \begin{pmatrix} | & | & \dots & | \\ \mathbf{b}_1 & \mathbf{b}_2 & \dots & \mathbf{b}_q \\ | & | & & | \end{pmatrix}$$

- **Vytvoření transformovaného datasetu:**

- Transformovaný dataset $\mathbf{T}_q$ vytvoříme jako:

$$\mathbf{T}_q = \mathbf{X}' \mathbf{V}$$

Hlavní komponenty a jejich statistická interpretace

- Uvažujme T_i jako i -tý sloupec matice $\mathbf{T}_q$, tedy:

$$T_i = \mathbf{b}_i^T \mathbf{X}'$$

- Tyto nové příznaky (sloupce T_i) nazýváme **hlavní komponenty**.
- **Statistická interpretace:**

- Co se stane s **průměrem**?

$$\bar{T}_i = \frac{1}{N} \sum_{k=1}^N \mathbf{b}_i^T \mathbf{x}'_k = \mathbf{b}_i^T \left(\frac{1}{N} \sum_{k=1}^N \mathbf{x}'_k \right) = 0$$

zůstane stále 0.

- Jak je na tom nová **varianční matice**?

$$\Sigma_T = \frac{1}{N-1} \mathbf{T}_q^T \mathbf{T}_q$$

tedy

$$\begin{aligned} \widehat{\text{cov}}(T_i, T_j) &= \frac{1}{N-1} (\mathbf{T}_q^T \mathbf{T}_q)_{i,j} \\ &= \frac{1}{N-1} (\mathbf{V}^T \mathbf{X}'^T \mathbf{X}' \mathbf{V})_{i,j} \\ &= \frac{1}{N-1} \mathbf{b}_i^T \mathbf{X}'^T \mathbf{X}' \mathbf{b}_j \\ &= \lambda_j \mathbf{b}_i^T \mathbf{b}_j = \begin{cases} 0 & \text{pro } i \neq j, \\ \lambda_j & \text{pro } i = j. \end{cases} \end{aligned}$$

- Platí tedy:
 - Rozptyl i -té hlavní komponenty je roven i -tému vlastnímu číslu λ_i .
 - Výběrem q hlavních komponent tedy vybereme směry, ve kterých mají data **největší rozptyl**.
 - Různé komponenty jsou **nekorelované**.
- **Závěrečné poznámky:**
 - Zvolením $q = p$ nezahodíme žádnou informaci, ale pouze přejdeme do **reprezentace dat v ortonormální bázi**.
 - Pro spektrální rozklad se používá SVD rozklad, který je numericky stabilnější.
 - PCA se soustředí na **globální vlastnosti** namísto **lokálních vztahů**.

6. Manifold learning a lokálně lineární vnoření

Problematika vysokých dimenzí

viz. Otázka 4.

Hypotéza variet

viz Otázka 4.

Lokálně lineární vnoření

- Sleduje, jak jednotlivé body z trénovací množiny **závisí lineárně na svém nejbližším okolí**.
- Následně **hledá méně dimenzionální reprezentaci**, která tyto **vztahy zachovává**.
- **Postup:**
 - Uvažujeme i -tý body trénovací množiny $\mathbf{x}_i$.
 - Pro dané k nalezneme jeho k nejbližších sousedů.
 - Dále najdeme váhy $w_{i,j}$ takové, že:
 - $w_{i,j} = 0$ pro j takové, že $\mathbf{x}_j$ není mezi k nejbližšími sousedy $\mathbf{x}_i$.
 - $\sum_j w_{i,j} = 1$,
 - vzdálenost mezi $\mathbf{x}_i$ a $\sum_j w_{i,j} \mathbf{x}_j$ byla nejmenší možná.

- Toto uděláme pro všechny body trénovací množiny. Ekvivalentně hledáme:

$$\arg \min_{w_{i,j}} \sum_{i=1}^N \left\| \mathbf{x}_i - \sum_j w_{i,j} \mathbf{x}_j \right\|^2$$

- Označme matici $\mathbf{W}^*$ takovou, že obsahuje optimální hodnoty vah $w_{i,j}^*$.
- V dalším kroku hledáme pro každé $\mathbf{x}_i \in \mathbb{R}^p$ q rozměrnou reprezentaci $\mathbf{z}_i \in \mathbb{R}^q$ takovou, že je vzdálenost mezi $\mathbf{z}_i$ a $\sum_j w_{i,j}^* \mathbf{z}_j$ nejmenší možná.
- Takto získáme matici $\mathbf{Z}^*$, která má v řádcích zapsané výsledné optimální hodnoty $\mathbf{z}_i^*$.

- **Shrnutí:**

- **Nalezení vah:**

$$\mathbf{W}^* = \arg \min_{\{w_{i,j}\}} \sum_{i=1}^N \left\| \mathbf{x}_i - \sum_j w_{i,j} \mathbf{x}_j \right\|^2$$

Lidsky: Hledáme množinu všech optimálních vah $w_{i,j}^*$.

- **Nalezení vnoření:**

$$\mathbf{Z}^* = \arg \min_{\{\mathbf{z}_i\}} \sum_{i=1}^N \left\| \mathbf{z}_i - \sum_j w_{i,j} \mathbf{z}_j \right\|^2$$

Lidsky: Hledáme množinu všech nových vektorů $\mathbf{z}_i^*$, přičemž používáme už nalezené optimální váhy $w_{i,j}^*$.

7. Naivní Bayesův klasifikátor

Klasifikace pomocí podmíněné pravděpodobnosti

- Chceme predikovat $P(Y = y | \mathbf{X} = \mathbf{x})$.
- Místo přímé konstrukce ale použijeme **Bayesovu větu**:

$$P(Y = y | \mathbf{X} = \mathbf{x}) = \frac{P(\mathbf{X} = \mathbf{x} | Y = y)P(Y = y)}{P(\mathbf{X} = \mathbf{x})}$$

- Tedy *nějak* odhadneme $P(\mathbf{X} = \mathbf{x} | Y = y)$.
- Jak na to:
 - Zajímá nás argument maxima vzhledem k y , tedy zahodíme $P(\mathbf{X} = \mathbf{x})$. (Je pro všechny hodnoty y stejný.)
 - Pro predikci tedy dostáváme:

$$\hat{Y} = \arg \max_{y \in \mathcal{Y}} P(\mathbf{X} = \mathbf{x} | Y = y)P(Y = y)$$

- Tomuto odhadu se říká **MAP odhad** (z angl. **m**aximum **a** **p**osteriori **e**stimate).
- Ted' už jen *nějak* odhadneme $P(\mathbf{X} = \mathbf{x} | Y = y)$.

Naivní Bayesův klasifikátor

- **Předpoklad:**
 - **Za podmínky** $Y = y$ jsou všechny příznaky nezávislé.
- Z tohoto předpokladu plyne, že:

$$P(\mathbf{X} = \mathbf{x} | Y = y) = \prod_{i=1}^p P(X_i = x_i | Y = y) P(Y = y)$$

- **Naivita** plyne z toho, že předpokládáme, že pro **fixní hodnotu** Y jsou příznaky nezávislé.
- Výsledný **MAP odhad** naivního Bayesova klasifikátoru je tedy:

$$\hat{Y} = \arg \max_{y \in \mathcal{Y}} \prod_{i=1}^p P(X_i = x_i | Y = y) P(Y = y)$$

- *Poznámka: Ačkoliv je v praxi zmíněný předpoklad velmi hrubý a naivní, ukazuje se, že model v mnoha případech dává až překvapivě dobré výsledky.*

Modely podmíněných pravděpodobností

- **Binární veličina X :**
 - V tomto případě je vhodný model pro rozdělení $X|Y = y$ **Bernoulliho rozdělení** s parametrem p_y .
 - Označujeme $(X|Y = y) \sim Be(p_y)$.
 - Pro pravděpodobnosti platí:
 $P(X = 1 | Y = y) = p_y$ a $P(X = 0 | Y = y) = 1 - p_y$
 - Odhad parametru p_y :

$$\hat{p}_y = \frac{N_{1,y}}{N_{1,y} + N_{0,y}}$$

kde $N_{i,y}$ značí počet dat pro $X = i$ a $Y = y$.

- Jedná se o **maximálně věrohodnostní odhad (MLE)**.
- Je však jasné, že pokud dojde k $N_{0,y} = 0$ nebo $N_{1,y} = 0$, celá pravděpodobnost zdegeneruje na $\hat{p}_y = 0$ (resp. 1).
- To ale znamená násobení nulou v MAP odhadu a celý odhad spadne na nulu. *Zero-frequency problem*.
- To se dá vyřešit **Bayesovským přístupem**:
Ve zkratce:
 - Zavedeme **apriorní rozdělení** - jak si myslíme, že bude parameter p_y vypadat. (U mince, třeba 1/2).
 - Na základě napozorovaných dat pak určíme **aposteriorní rozdělení**:

$$f_p(p|\mathbf{x}) = \frac{P(\mathbf{X} = \mathbf{x}|p) f_p(p)}{P(\mathbf{X} = \mathbf{x})},$$

kde $P(\mathbf{X} = \mathbf{x})$ je pravděpodobnost, že napozorujeme $\mathbf{X} = \mathbf{x}$ pokud p je správný parametr a

$$P(\mathbf{X} = \mathbf{x}) = \int_p P(\mathbf{X} = \mathbf{x}|p) f_p(p) dp$$

- Bodový odhad p se získá spočtením střední hodnoty:

$$\hat{p} = \int_p p f_p(p|\mathbf{x}) dp.$$

- V případě Bernoulliho rozdělení se bere jako apriorní rozdělení **Beta rozdělení**.
- Výsledný odhad z apriorního rozdělení je pak:

$$\hat{p}_y = \frac{N_{1,y} + 1}{N_{1,y} + N_{0,y} + 2}$$

- Tomuto se říká **add-one smoothing (Laplace smoothing)**.

- **Kategorická veličina X :**

- Nechť X nabývá k různých hodnot $c_1, \dots, c_k$.
- Vhodným modelem je potom **kategorické rozdělení**.
- Zde máme $\mathbf{p}_y = (p_{1,y}, \dots, p_{k,y})^T$, kde $p_{1,y}, \dots, p_{k,y}$ jsou podmíněné pravděpodobnosti hodnot $c_1, \dots, c_k$.
- **Odhad k -rozměrného parametru $\mathbf{p}_y$ se využívá:**

$$\hat{\mathbf{p}}_y = (\hat{p}_{1,y}, \dots, \hat{p}_{k,y})^T \quad \text{a} \quad \hat{p}_{j,y} = \frac{N_{j,y} + 1}{N_{1,y} + \dots + N_{k,y} + k}$$

- **Spojité veličina X :**

- Místo podmíněné pravděpodobnosti se vezme **podmíněná hustota** $f_{X|y}(x)$.
- Této hustotě odpovídá **distribuční funkce**:

$$F_{X|y}(x) = P(X \leq x | Y = y).$$

- Predikci provedeme pomocí MAP odhadu:

$$\hat{Y} = \arg \max_{y \in \mathcal{Y}} \prod_{i=1}^l P(X_i = x_i | Y = y) \prod_{i=l+1}^p f_{X_i|y}(x_i) P(Y = y),$$

kde $X_1, \dots, X_l$ jsou diskrétní příznaky a $X_{l+1}, \dots, X_p$ jsou spojité příznaky.

- **Gaussovo rozdělení:**

- Předpokládejme, že $X|Y = y \sim \mathcal{N}(\mu_y, \sigma_y^2)$.
- Použijeme MLE odhady:

$$\hat{\mu}_y = \frac{1}{N_y} \sum_{i=1}^{N_y} x_i \quad \text{a} \quad \hat{\sigma}_y^2 = \frac{1}{N_y} \sum_{i=1}^{N_y} (x_i - \hat{\mu}_y)^2,$$

kde x_i jsou hodnoty příznaku X , pro které $Y = y$.

8. Lineární diskriminační analýza (LDA)

Generativní versus diskriminativní modely

- **Generativní model:**

- Odhadujeme podmíněnou pravděpodobnost $P(\mathbf{X} = \mathbf{x} | Y = y)$.
- Společně pak s $P(Y = y)$ dostáváme sdruženou pravděpodobnost:

$$P(\mathbf{X} = \mathbf{x}, Y = y) = P(\mathbf{X} = \mathbf{x} | Y = y)P(Y = y).$$

- Tento přístup, kdy vytváříme sdruženou pravděpodobnost $P(\mathbf{X} = \mathbf{x}, Y = y)$ se nazývá **generativní přístup** a výsledný model je pak **generativní model**.
- Známe úplnou informaci o rozdělení, ze kterého byla data generována a jsme schopni generovat sami nová data.

- **Diskriminativní model:**

- Druhou možností je odhadovat z trénovacího datasetu podmíněnou pravděpodobnost $P(Y = y | \mathbf{X} = \mathbf{x})$.
- Tomuto přístupu se říká **diskriminativní**.

Log-sum-exp trik

- Počítat

$$P(Y = y | \mathbf{X} = \mathbf{x}) = \frac{P(\mathbf{X} = \mathbf{x} | Y = y)P(Y = y)}{\sum_{y \in \mathcal{Y}} P(\mathbf{X} = \mathbf{x} | Y = y)P(Y = y)}$$

může vést k **numerickému podtečení**.

- Proto spočítáme logaritmus celé pravděpodobnosti

$$\begin{aligned} \ln P(Y = y | \mathbf{X} = \mathbf{x}) &= \ln P(\mathbf{X} = \mathbf{x} | Y = y) + \ln P(Y = y) \\ &\quad - \ln \sum_{y \in \mathcal{Y}} P(\mathbf{X} = \mathbf{x} | Y = y)P(Y = y) \end{aligned}$$

- Uděláme následující úpravu

$$\ln \sum_{y \in \mathcal{Y}} P(\mathbf{X} = \mathbf{x} | Y = y)P(Y = y) = \ln \sum_{y \in \mathcal{Y}} e^{-b_y},$$

kde $b_y = -\ln P(\mathbf{X} = \mathbf{x} | Y = y) - \ln P(Y = y)$

- **Log-sum-exp trik:**

$$\ln(e^{-120} + e^{-121}) = \ln(e^{-120}(e^0 + e^{-1})) = \ln(e^0 + e^{-1}) - 120$$

což už je numericky stabilní.

- V našem případě dostaneme:

$$\begin{aligned} \ln \sum_{y \in \mathcal{Y}} e^{-b_y} &= \\ \ln \sum_{y \in \mathcal{Y}} e^{-B} e^{B-b_y} &= \\ -B + \ln \sum_{y \in \mathcal{Y}} e^{B-b_y} \end{aligned}$$

kde $B = \min_{y \in \mathcal{Y}} b_y$.

(Gaussovská) diskriminační analýza

- Modelujeme pomocí **vícerozměrného normálního rozdělení**.

- **Definice:**

Bud' $\boldsymbol{\mu} \in \mathbb{R}^p$ vektor středních hodnot a $\boldsymbol{\Sigma} \in \mathbb{R}^{p,p}$ symetrická a **pozitivně semidefinitní** matice. Náhodný vektor $\mathbf{X} = (X_1, \dots, X_p)^T$ má **p -rozměrné normální rozdělení** s parametry $\boldsymbol{\mu}$ a $\boldsymbol{\Sigma}$, píšeme $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, jestliže pro každé $\mathbf{c} \in \mathbb{R}^p$ platí $\mathbf{c}^T \mathbf{X} \sim \mathcal{N}(\mathbf{c}^T \boldsymbol{\mu}, \mathbf{c}^T \boldsymbol{\Sigma} \mathbf{c})$.

- **Věta:**

Bud' $\boldsymbol{\mu} \in \mathbb{R}^p$ vektor a $\boldsymbol{\Sigma} \in \mathbb{R}^{p,p}$ symetrická a **pozitivně definitní** matice. Potom $\mathbf{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ právě tehdy, když $\mathbf{X}$ má *spojité sdružené rozdělení s hustotou*:

$$f_{\mathbf{X}}(\mathbf{x}) = \frac{1}{(2\pi)^{p/2} |\boldsymbol{\Sigma}|^{1/2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$

- Podmíněné sdružené rozdělení tedy modelujeme vícerozměrným normálním rozdělením:

$$\mathbf{X}|Y = y \sim \mathcal{N}(\boldsymbol{\mu}_y, \boldsymbol{\Sigma}_y)$$

- MAP odhad je dán vztahem:

$$\hat{Y} = \arg \max_{y \in \mathcal{Y}} f_{\mathbf{X}|y}(\mathbf{x}) P(Y = y).$$

Lineární diskriminační analýza

- **Kvadratická diskriminační analýza:**

- MAP odhad Y maximalizuje pravděpodobnosti

$$P(Y = y | \mathbf{X} = \mathbf{x}) = \frac{f_{\mathbf{X}|y}(\mathbf{x}) P(Y = y)}{\sum_{y \in \mathcal{Y}} f_{\mathbf{X}|y}(\mathbf{x}) P(Y = y)}.$$

- Dosadíme-li předpis pro sdruženou hustotu, dostaneme:

$$P(Y = y | \mathbf{X} = \mathbf{x}) = \frac{\pi_y (2\pi)^{-\frac{p}{2}} |\boldsymbol{\Sigma}_y|^{-\frac{1}{2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_y)^T \boldsymbol{\Sigma}_y^{-1}(\mathbf{x}-\boldsymbol{\mu}_y)}}{\sum_{c \in \mathcal{Y}} \pi_c (2\pi)^{-\frac{p}{2}} |\boldsymbol{\Sigma}_c|^{-\frac{1}{2}} e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}_c)^T \boldsymbol{\Sigma}_c^{-1}(\mathbf{x}-\boldsymbol{\mu}_c)}},$$

kde $\pi_y = P(Y = y)$.

- Při MAP odhadu pro y s největší pravděpodobností se porovnávají velikosti čitatelů a v důsledku z pohledu x tedy hodnoty **kvadratické** členy $(x - \mu_y)^T \Sigma_y^{-1} (x - \mu_y)$.
- **Lineární diskriminační analýza:**
 - Je speciální případ **kvadratické diskriminační analýzy**, kde předpokládáme, že **varianční matice jsou pro všechny třídy stejné: $\Sigma_y = \Sigma$** .
 - Tedy pro MAP odhad máme v čitateli:

$$\begin{aligned} \pi_y (2\pi)^{-\frac{p}{2}} |\Sigma|^{-\frac{1}{2}} e^{-\frac{1}{2} (x - \mu_y)^T \Sigma^{-1} (x - \mu_y)} &= \\ \pi_y (2\pi)^{-\frac{p}{2}} |\Sigma|^{-\frac{1}{2}} e^{-\frac{1}{2} (-2\mu_y^T \Sigma^{-1} x + x^T \Sigma^{-1} x + \mu_y^T \Sigma^{-1} \mu_y)} &= \\ (2\pi)^{-\frac{p}{2}} |\Sigma|^{-\frac{1}{2}} e^{-\frac{1}{2} x^T \Sigma^{-1} x} e^{(\mu_y^T \Sigma^{-1} x - \frac{1}{2} \mu_y^T \Sigma^{-1} \mu_y + \ln \pi_y)} & \end{aligned}$$

- Člen $(2\pi)^{-\frac{p}{2}} |\Sigma|^{-\frac{1}{2}} e^{-\frac{1}{2} x^T \Sigma^{-1} x}$ nezávisí na y a ve zlomku se zkrátí.
- Zavedeme:

$$\gamma_y = -\frac{1}{2} \mu_y^T \Sigma^{-1} \mu_y + \ln \pi_y \quad \text{a} \quad \beta_y = \Sigma^{-1} \mu_y$$

a můžeme psát:

$$P(Y = y | \mathbf{X} = \mathbf{x}) = \frac{e^{\beta_y^T \mathbf{x} + \gamma_y}}{\sum_{c \in \mathcal{Y}} e^{\beta_c^T \mathbf{x} + \gamma_c}}.$$

- z MAP pohledu se jedná o porovnání čitatelů a v důsledku toho porovnání výrazů **lineárních** v x .
- Pro rozhodovací hranici mezi třídami y a y' platí:

$$\begin{aligned} P(Y = y | \mathbf{X} = \mathbf{x}) &= P(Y = y' | \mathbf{X} = \mathbf{x}) \\ \beta_y^T \mathbf{x} + \gamma_y &= \beta_{y'}^T \mathbf{x} + \gamma_{y'} \\ (\beta_y - \beta_{y'})^T \mathbf{x} &= \gamma_{y'} - \gamma_y \end{aligned}$$

je tedy určena nadrovinou v $\mathbb{R}^p$.

- Při binární klasifikaci dostaneme:

$$P(Y = 1 | \mathbf{X} = \mathbf{x}) = \frac{e^{\beta_1^T \mathbf{x} + \gamma_1}}{e^{\beta_1^T \mathbf{x} + \gamma_1} + e^{\beta_0^T \mathbf{x} + \gamma_0}} = \sigma(\mathbf{w}^T \mathbf{x} + w_0),$$

kde σ je logistická funkce s $\mathbf{w} = \beta_1 - \beta_0$ a $w_0 = \gamma_1 - \gamma_0$.

- Pro odhad parametrů opět využíváme MLE:

$$\hat{\pi}_y = \frac{N_y}{N}, \quad \hat{\mu}_y = \frac{1}{N_y} \sum_{\{i|y_i=y\}} \mathbf{x}_i.$$

$$\hat{\Sigma} = \frac{1}{N} \left(\sum_{\{i|y_i=0\}} (\mathbf{x}_i - \hat{\mu}_0)(\mathbf{x}_i - \hat{\mu}_0)^T + \sum_{\{i|y_i=1\}} (\mathbf{x}_i - \hat{\mu}_1)(\mathbf{x}_i - \hat{\mu}_1)^T \right)$$

Fisherova lineární diskriminační analýza

- Budeme chápat $\sigma(\mathbf{w}^T \mathbf{x} + w_0)$ jako **ortogonální projekci** do směru $\mathbf{w}$ posunutou o w_0 .

- Cílem je udělat **projekci na podprostor menší dimenze** tak, aby transformovaná data byla co **nejpříznivější pro predikci**.
- Od LDA se liší tím, že není formulován generativním přístupem a tedy neobsahuje požadavky na rozdělení.
- **Binární klasifikace:**
 - Jednorozměrný podprostor můžeme určit vektorem $\mathbf{w}$ a ortogonální projekci bodu $\mathbf{x}_i$ na něj pak odpovídá transformaci

$$z_i = \frac{\mathbf{w}^T \mathbf{x}_i}{\|\mathbf{w}\|}.$$

- Dále zavedeme projekce výběrových průměrů (geometrických středů) $\hat{\boldsymbol{\mu}}_y$ z jednotlivých tříd jako $m_y = \frac{1}{\|\mathbf{w}\|} \mathbf{w}^T \hat{\boldsymbol{\mu}}_y$.
- Rozptyl projektovaných bodů v rámci jedné třídy můžeme (až na konstantu) vyjádřit jako

$$s_y^2 = \sum_{\{i|y_i=y\}} (z_i - m_y)^2.$$

- Cílem FLDA je najít vektor $\mathbf{w}$, který **maximalizuje vzdálenost** mezi průměry projektovaných tříd a zároveň drží **rozptyl** obou tříd **malé**.
- Toto můžeme vyjádřit pomocí účelové funkce

$$J(\mathbf{w}) = \frac{(m_1 - m_0)^2}{s_1^2 + s_0^2}$$

- Když zavedeme **rozptylovou matici mezi třídami**

$$\mathbf{S}_B = (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0)(\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0)^T$$

- a **rozptylovou matici v rámci třídy**

$$\mathbf{S}_W = \sum_{\{i|y_i=0\}} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_0)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_0)^T + \sum_{\{i|y_i=1\}} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_1)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_1)^T,$$

- tak můžeme získat účelovou funkcí jako

$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}}$$

- Pomocí gradientu pak lze ukázat, že maximum nastává, když

$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}, \quad \text{kde } \lambda = J(\mathbf{w})$$

- což dává rovnici pro vlastní vektor

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w}$$

- Lze vidět, že

$$\mathbf{S}_B \mathbf{w} = (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0)(m_1 - m_0) \|\mathbf{w}\|$$

- Dosazením do předchozího vztahu

$$\lambda \mathbf{w} = \mathbf{S}_W^{-1} (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0)(m_1 - m_0) \|\mathbf{w}\|$$

- což až na konstantu je rovno

$$\mathbf{w} = N \mathbf{S}_W^{-1} (\hat{\boldsymbol{\mu}}_1 - \hat{\boldsymbol{\mu}}_0)$$

- To je **totožný vztah jako u LDA**, kde $\mathbf{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)$, protože $\mathbf{S}_W = N \hat{\boldsymbol{\Sigma}}$

- **Klasifikace do více tříd:**

- V případě C tříd hledáme matici $\mathbf{W}$.
- Účelová funkce je

$$J(\mathbf{w}) = \frac{\det(\mathbf{W}^T \mathbf{S}_B \mathbf{W})}{\det(\mathbf{W}^T \mathbf{S}_W \mathbf{W})}$$

- Rozptylové matice mezi třídami i rozptylové matice v rámci třídy jsou definovány následovně

$$\mathbf{S}_B = \sum_{y \in \mathcal{Y}} (\hat{\boldsymbol{\mu}}_y - \hat{\boldsymbol{\mu}})(\hat{\boldsymbol{\mu}}_y - \hat{\boldsymbol{\mu}})^T$$

$$\mathbf{S}_W = \sum_{y \in \mathcal{Y}} \sum_{\{i | y_i = y\}} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_y)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_y)^T$$

9. Perceptron

Definice perceptronu

- **Definice:**

Perceptron je nejjednodušší model dopředné neuronové sítě. Sestává pouze z jednoho neuronu.

Na vstupu mějme vektor $\mathbf{x} = (x_1, \dots, x_p)$, k němu příslušné váhy $\mathbf{w} = (w_1, \dots, w_p)$ a *intercept (bias)* w_0 .

Výstup perceptronu se získá aplikací **skokové aktivační funkce** g na **vnitřní potenciál** ξ .

Skoková aktivační funkce:

$$g(\xi) = \begin{cases} 1 & \text{pro } \xi \geq 0 \\ 0 & \text{pro } \xi < 0 \end{cases}$$

Vnitřní potenciál:

$$\xi = w_0 + \sum_{i=1}^p w_i x_i = \mathbf{w}^T \mathbf{x} + w_0$$

Neuron je tedy aktivován, pokud $\mathbf{w}^T \mathbf{x} \geq -w_0$, $-w_0$ se někdy nazývá **prahová hodnota**.

Algoritmus trénování perceptronu a jeho konvergence

• Algoritmus trénování

- Postupně procházíme body trénovací množiny
- Pro daný bod uděláme predikci a provedeme:

$$\begin{aligned} \text{error} &= Y - \hat{Y}, \\ w_i &\leftarrow w_i + \text{error} \cdot x_i, \quad i = 1, \dots, p, \\ w_0 &\leftarrow w_0 + \text{error}, \end{aligned}$$

kde $\hat{Y} = g(\mathbf{w}^T \mathbf{x} + w_0)$ je predikce v bodě $\mathbf{x}$ a Y je skutečná hodnota.

- Algoritmus ukončíme, jakmile nemáme chybu pro žádný bod.
- **Věta o konvergenci**

Předpokládejme lineárně separabilní data, tj.

$$(\exists \tilde{\mathbf{w}}^* \in \mathbb{R}^{p+1}, \|\tilde{\mathbf{w}}^*\| = 1)(\exists \gamma > 0)(\forall i \in \{1, \dots, N\}) : \\ (2Y_i - 1)\tilde{\mathbf{x}}_i^T \tilde{\mathbf{w}}^* > \gamma$$

Dále předpokládejme, že pro nějaké $R > 0$ platí $\|\tilde{\mathbf{x}}_i\| \leq R$ pro všechna $i = 1, \dots, N$.
Potom počet kroků algoritmu trénování perceptronu, při kterých nastane chyba, je nejvýše $\frac{R^2}{\gamma^2}$.

Poznámky a značení:

$(\exists \tilde{\mathbf{w}}^* \in \mathbb{R}^{p+1}, \|\tilde{\mathbf{w}}^*\| = 1)$: $\tilde{\mathbf{w}}^* = (w_0, w_1, \dots, w_p)^T$, tedy hledáme (zda existuje) **vektor vah + bias, jednotkové délky**, pro který bude platit ta podmínka.

$(\exists \gamma > 0)$: Hledáme $\gamma > 0$, to představuje **šířku mezery** mezi třídami. Proto $\gamma > 0$.

$(\forall i \in \{1, \dots, N\})$: N je počet bodů trénovací množiny, tedy budeme kvantifikovat **pro všechny** body trénovací množiny.

$(2Y_i - 1)\tilde{\mathbf{x}}_i^T \tilde{\mathbf{w}}^* > \gamma$: $(2Y_i - 1)$ pouze převádí hodnoty z $Y_i \in \{0, 1\} \rightarrow \{-1, 1\}$. Výraz $\tilde{\mathbf{x}}_i^T \tilde{\mathbf{w}}^*$ představuje hodnotu vnitřního potenciálu pro ideální váhy, a protože $\|\tilde{\mathbf{w}}^*\| = 1$, představuje tento součin **geometrickou vzdálenost** bodu $\tilde{\mathbf{x}}_i$ od rozhodovací hranice. A tato vzdálenost od rozhodovací hranice musí být větší γ (nějaké kladné číslo). Bod tedy nesmí ležet přímo na rozhodovací hranici.

$(\exists R > 0)(\|\tilde{\mathbf{x}}_i\| \leq R)$: Můžeme si představit jako **poloměr nejmenší možné koule** se středem v $\mathbf{0}$, do které se vejdou všechny trénovací body.

$\frac{R^2}{\gamma^2}$: Výsledný zlomek nám udává, *jak dlouho (nejdéle) potrvá trénování*.

Čím **větší** R (čím více jsou data *roztahána* od středu), tím **déle trénování potrvá**.

Naopak čím **větší** γ (čím dále jsou od sebe třídy), tím **kratší dobu trénování potrvá**.

Když zase γ bude **malé**, třídy jsou k sobě blízko a **trénování** tedy **potrvá déle**.

Problémy perceptronu

- Dokáže řešit pouze **lineárně separabilní** problémy.
- Jednoduchý příklad, který perceptron nedokáže vyřešit, je funkce **XOR**.
- K vyřešení XORu stačí přitom vzít tři perceptrony, tomuto přístupu pak odpovídá použití **vícevrstvé neuronové sítě**.

10. Vícevrstvé neuronové sítě - aktivační funkce, dopředný a zpětný chod

- Vícevrstvé neuronové sítě jsou základním rozšířením jednoho neuronu.

Matematický popis vícevrstvé neuronové sítě

- Označíme $n_1, \dots, n_l$ počty neuronů v jednotlivých vrstvách.
- Na i -té vrstvě mějme výstup j -tého neuronu: $f_j^{(i)} : \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}$.
- $f_j^{(i)}$ se opět počítá jako $g(\mathbf{w}^T \mathbf{x} + w_0)$, kde g je aktivační funkce daného neuronu a $w_0, w_1, \dots, w_{n-1}$ jsou jeho váhy.
- i -tou vrstvu pak chápeme jako vícehodnotovou funkci $\mathbf{f}^{(i)} : \mathbb{R}^{n_{i-1}} \rightarrow \mathbb{R}^{n_i}$, kde $\mathbf{f}^{(i)} = (f_1^{(i)}, \dots, f_{n_i}^{(i)})^T$.
- **Celá neuronová síť** je pak reprezentovaná funkcí $\mathbf{f} : \mathbb{R}^{n_0} \rightarrow \mathbb{R}^{n_l}$

$$\mathbf{f} = \mathbf{f}^{(l)} \circ \mathbf{f}^{(l-1)} \circ \dots \circ \mathbf{f}^{(2)} \circ \mathbf{f}^{(1)}.$$

zkráceně

$$\bigcirc_{i=1}^l \mathbf{f}^{(i)} \quad \text{nebo} \quad \bigcirc_{i=1}^l \mathbf{f}^{(l-i+1)}$$

Aktivační funkce pro výstupní i skryté vrstvy

- **Skryté vrstvy**
 - V případě i -té skryté vrstvy se nejdříve spočítá vektor lineárních kombinací z vektoru vstupů z předchozí vrstvy

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{w}_0.$$

- Potom se na každou složku z_i aplikuje nelineární aktivační funkce, která je skoro všude diferencovatelná (a hlavně někde nemá nulový gradient).
- Nejčastější příklady:
 - **ReLU** (rectified linear unit)

$$g(z) = \max(0, z)$$

- **Leaky ReLU**

$$g(z) = \max(0.01z, z)$$

na rozdíl od od ReLU má nenulový gradient v záporných hodnotách.

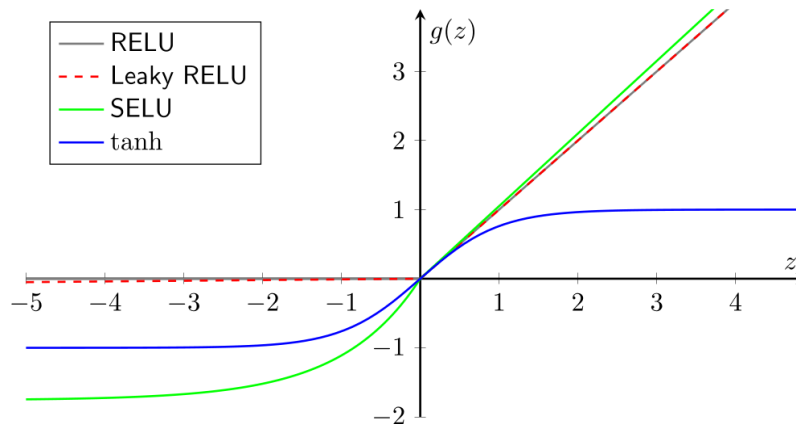
- **SELU** (scaled exponential linear unit)

$$g(z) = \lambda \begin{cases} z & \text{pro } z \geq 0, \\ \alpha(e^z - 1) & \text{pro } z < 0 \end{cases}$$

- **Hyperbolický tangens**

$$g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Nejpoužívanější aktivační funkce



- Poslední skrytá vrstva produkuje vektor n_{l-1} skrytých příznaků $\mathbf{h}$.
- Na výstupní vrstvě se pak spočítá vektor lineárních kombinací

$$\mathbf{z} = \mathbf{W}^T \mathbf{h} + \mathbf{w}_0$$

kde i -tý sloupec matice $\mathbf{W} \in \mathbb{R}^{n_{l-1}, n_l}$ a i -tá složka $\mathbf{w}_0 \in \mathbb{R}^l$ jsou váhy i -tého neuronu.

- Poté se na $\mathbf{z}$ aplikuje výstupní aktivační funkce .
- **Výstupní vrstva**

Podle typu úlohy aplikujeme vhodnou aktivační funkci

- **Regresní úloha**

- Ve výstupní vrstvě použijeme jeden neuron s **aktivační funkcí identity**

$$g(z) = z.$$

- Výstupem je tedy

$$\hat{Y} = \mathbf{w}^T \mathbf{h} + w_0.$$

- **Binární klasifikace**

- Ve výstupní vrstvě se použije jeden neuron s **aktivační funkcí logistické sigmoidy**

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- Výstup potom, stejně jako u logistické regrese, interpretujeme jako pravděpodobnost příslušnosti ke třídě 1

$$\hat{P}(Y = 1 | \mathbf{X} = \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + w_0).$$

- Predikce v bodě $\mathbf{x}$ je potom

$$\hat{Y} = \begin{cases} 1 & \text{pokud } \hat{P}(Y = 1 | \mathbf{X} = \mathbf{x}) > \frac{1}{2}, \\ 0 & \text{jinak.} \end{cases}$$

- **Klasifikace do c tříd**

- Ve výstupní vrstvě se používá c neuronů s normovanou **aktivační funkcí softmax**

$$\text{softmax}_i(\mathbf{z}) = \frac{e^{z_i}}{\sum_{i=1}^c e^{z_i}}.$$

- Výstupní hodnota $\text{softmax}_i(\mathbf{z})$ i -tého neuronu interpretujeme jako pravděpodobnost příslušnosti ke třídě i , tj.

$$\hat{P}(Y = i | \mathbf{X} = \mathbf{x}) = \text{softmax}_i(\mathbf{W}^T \mathbf{x} + \mathbf{w}_0).$$

- K predikci je vybrána třída s největší pravděpodobností

$$\hat{Y} = \arg \max_{i \in \{1, \dots, c\}} \hat{P}(Y = i | \mathbf{X} = \mathbf{x}).$$

Používané ztrátové funkce

- **Regresní úloha**

Typickou volbou je **squared error**

$$L(Y, \hat{Y}) = (Y - \hat{Y})^2.$$

Další možností je L_1 ztrátová funkce **absolute error**

$$L(Y, \hat{Y}) = |Y - \hat{Y}|.$$

- **Binární úloha**

Používá se **binary cross-entropy loss**

$$L(Y, \hat{p}) = -Y \log \hat{p} - (1 - Y) \log(1 - \hat{p}).$$

- **Kategorická úloha**

Používá se **categorical cross-entropy loss**

$$L(Y, \hat{\mathbf{p}}) = - \sum_{j=1}^c \mathbb{1}_{Y=j} \log \hat{p}_j = - \log \hat{p}_Y.$$

- **Účelová funkce**

- U **regrese** se akorát přidá *mean*, tedy: **mean squared error** a sečte se ztrátová funkce přes všechny body datasetu a zprůměruje se.

- U **binární/klasifikační úlohy** se akorát zahodí slovo *loss* a opět se pro každý bod sečte jeho ztrátová funkce a zprůměruje se.

Dopředný a zpětný chod

- Minimalizujeme účelovou funkci $J(\mathbf{w})$ pomocí gradientního sestupu.
 - Při výpočtu gradientu používáme pravidlo pro **derivaci složené vícerozměrné funkce**.
 - Pro složení $\mathbf{g} = (g_1, \dots, g_n)^T, \mathbf{g} : \mathbb{R}^m \rightarrow \mathbb{R}^n$ s funkcí $f : \mathbb{R}^n \rightarrow \mathbb{R}$ se využívá **řetězové pravidlo**

$$\frac{\partial f \circ \mathbf{g}}{\partial x_j}(\mathbf{x}) = \sum_{i=1}^n \frac{\partial f(\mathbf{g})}{\partial g_i}(\mathbf{g}(\mathbf{x})) \frac{\partial g_i}{\partial x_j}(\mathbf{x}).$$

- **Dopředný chod (forward pass)**
 - Výpočet výstupu sítě v bodě $\mathbf{x}$ se nazývá dopředný chod.
 - Dopředný chod provedeme pro všechny body trénovací množiny a spočítáme účelovou funkci.
- **Zpětný chod (backward pass)**
 - Pro výpočet gradientu účelové funkce $J(\mathbf{w})$ podle $\mathbf{w}$ dostáváme j -tý člen

$$\frac{\partial J}{\partial w_j}(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^{n_l} \frac{\partial L(y, \mathbf{f})}{\partial f_k}(Y_i, \mathbf{f}(\mathbf{x}_i; \mathbf{w})) \frac{\partial f_k(\mathbf{w})}{\partial w_j}(\mathbf{x}_i; \mathbf{w})$$

- Pro výpočet parciální derivace $\frac{\partial f_k(\mathbf{w})}{\partial w_j}(\mathbf{x}_i; \mathbf{w})$ použijeme opět řetězové pravidlo a postupně bychom dostali

$$\begin{aligned} \frac{\partial f_k(\mathbf{w})}{\partial w_j}(\mathbf{x}_i; \mathbf{w}) &= \\ \sum_{m=1}^{n_{l-1}} \frac{\partial f_k^{(l)} \mathbf{f}^{(l-1)}}{\partial f_m^{(l-1)}}(\mathbf{f}^{(l-1)} \circ \dots \circ \mathbf{f}^{(1)}(\mathbf{x}; \mathbf{w})) &\frac{\partial (f_m^{(l-1)} \circ \dots \circ \mathbf{f}^{(1)})}{\partial w_j}(\mathbf{x}; \mathbf{w}) = \\ \sum_{m=1}^{n_{l-1}} \frac{\partial f_k^{(l)} \mathbf{f}^{(l-1)}}{\partial f_m^{(l-1)}} \sum_{k=1}^{n_{l-2}} \frac{\partial f_m^{(l)} \mathbf{f}^{(l-2)}}{\partial f_k^{(l-2)}} &\frac{\partial (f_m^{(l-2)} \circ \dots \circ \mathbf{f}^{(1)})}{\partial w_j}(\mathbf{x}; \mathbf{w}) \end{aligned}$$

- Toto řetězení aplikujeme dokud nenarazíme na vrstvu u a v ní neuron n , který přímo závisí na parametru w_j . V tento moment zastavíme výpočet.
- Celkově pak pro j -tou složku gradientu účelové funkce dostáváme

$$\begin{aligned} \frac{\partial J}{\partial w_j}(\mathbf{w}) &= \\ \frac{1}{N} \sum_{i=1}^N \sum_{k=1}^{n_l} \frac{\partial L(y, \mathbf{f})}{\partial f_k}(Y_i, \mathbf{f}(\mathbf{x}_i; \mathbf{w})) &\sum_{m=1}^{n_{l-1}} \frac{\partial f_k^{(l)} \mathbf{f}^{(l-1)}}{\partial f_m^{(l-1)}} \dots \frac{\partial f_n^{(u)} \mathbf{w}}{\partial w_j}(\mathbf{x}_i; \mathbf{w}). \end{aligned}$$

- Celý tento algoritmus nazýváme **zpětné šíření chyby** (angl. **back-propagation**).
- **Výpočetní graf**

- Pro výpočet dopředného i zpětného chodu se obvykle používají kvůli efektivitě **výpočetní grafy**.

11. Vícevrstvé neuronové sítě - dávkové učení, optimalizační metody

Formulace optimalizační úlohy učení

- Chceme minimalizovat chybu predikce pomocí **účelové funkce**

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N L(Y_i, \mathbf{f}(\mathbf{x}_i; \mathbf{w}))$$

vzhledem k parametrům $\mathbf{w}$.

- Pomocí **zpětného chodu** dostaneme $\nabla_{\mathbf{w}} J(\mathbf{w})$, který použijeme na update vah.
- Toto je **deterministická** metoda učení.
- U větších datasetů a velkého množství parametrů je ale výpočet zbytečně zdlouhavý a paměťově problematický.
- V jednom kroku se tedy využije jen **část trénovací množiny**.
- Gradient průměrné chyby na této části je pak **odhad** skutečného gradientu $\nabla_{\mathbf{w}} J(\mathbf{w})$.
- Této metodě se říká **dávkové učení**.

Dávkové učení

- V každém kroku je napočten gradient $\nabla_{\mathbf{w}} \hat{J}(\mathbf{w})$ účelové funkce

$$\hat{J}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m L(Y_{(i)}, \mathbf{f}(\mathbf{x}_i; \mathbf{w})),$$

kde velikost jedné dávky je m .

- Tento gradient se použije k updatu vah.
- Celý proces učení je tedy následující:
 - Náhodně promícháme body trénovací množiny.
 - Rozdělíme je na dávky o velikosti m .
 - Postupně **projdeme všechny dávky** a pro každou provedeme jeden krok gradientního sestupu.
 - Po $\lceil N/m \rceil$ jsme prošli celou množinu.
 - Jednomu tomuto průchodu říkáme **epocha**.
 - Následně pokračujeme další epochou.
 - Na konci každé epochy **vyhodnotíme stav** natrénování a změříme výkonnost na validační množině.
 - Trénování ukončíme po dosažení určitého počtu epoch nebo pokud aktuální stav odpovídá požadovanému.

Optimalizační metody - SGD (s hybností), RMSProp, Adam

- **SGD (s hybností):**

- Inicializujeme váhy $\mathbf{w}$ (a počáteční rychlost $\mathbf{v} = 0$).
- Zvolíme učicí parametr ϵ (a parametr hybnosti μ).
- **Dokud nejsou splněny podmínky zastavení:**
 - Vezmeme dávku bodů z trénovací množiny.
 - Spočítáme gradient: $\nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\mathbf{w}} L(Y_{(i)}, \mathbf{f}(\mathbf{x}_i; \mathbf{w}))$.
 - (Spočteme rychlost: $\mathbf{v} \leftarrow \mu \mathbf{v} - \epsilon \nabla_{\mathbf{w}} \hat{J}(\mathbf{w})$).
 - Provedeme update vah:

$$\mathbf{w} \leftarrow \mathbf{w} - \epsilon \nabla_{\mathbf{w}} \hat{J}(\mathbf{w}).$$

Nebo v případě s hybností:

$$\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}.$$

- **RMSProp:**

- Inicializujeme váhy $\mathbf{w}$, počáteční rychlost $\mathbf{v} = 0$ a akumulární proměnnou $\mathbf{r} = 0$.
- Zvolíme globální učicí parametr ϵ , parametr hybnosti μ , koeficient útlumu ρ a malou konstantu δ .
- **Dokud nejsou splněny podmínky zastavení:**
 - Vezmu dávku bodů z trénovací množiny.
 - Spočítáme gradient: $\nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\mathbf{w}} L(Y_{(i)}, \mathbf{f}(\mathbf{x}_i; \mathbf{w}))$.
 - Agregujeme kvadrát gradientu: $\mathbf{r} \leftarrow \rho \mathbf{r} + (1 - \rho) \nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) \odot \nabla_{\mathbf{w}} \hat{J}(\mathbf{w})$.
 - Spočteme rychlost: $\mathbf{v} \leftarrow \mu \mathbf{v} - \frac{\epsilon}{\delta + \sqrt{\mathbf{r}}} \odot \nabla_{\mathbf{w}} \hat{J}(\mathbf{w})$.
 - Provedeme update vah: $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$.

- **ADAM (ADAPtive Moments)**

- Inicializujeme váhy $\mathbf{w}$, akumulární proměnné $\mathbf{s} = 0$, $\mathbf{r} = 0$ a časový krok $t = 0$.
- Zvolíme globální učicí parametr ϵ , parametr hybnosti μ , koeficienty útlumu ρ_1, ρ_2 a malou konstantu δ .
- **Dokud nejsou splněny podmínky zastavení:**
 - Vezmeme dávku bodů z trénovací množiny.
 - Spočteme gradient.
 - Zvedneme časový krok: $t \leftarrow t + 1$.
 - Agregujeme první moment: $\mathbf{s} \leftarrow \rho_1 \mathbf{s} + (1 - \rho_1) \nabla_{\mathbf{w}} \hat{J}(\mathbf{w})$.
 - Agregujeme druhý moment: $\mathbf{r} \leftarrow \rho_2 \mathbf{r} + (1 - \rho_2) \nabla_{\mathbf{w}} \hat{J}(\mathbf{w}) \odot \nabla_{\mathbf{w}} \hat{J}(\mathbf{w})$.
 - Provedeme korekci obou momentů: $\hat{\mathbf{s}} \leftarrow \frac{\mathbf{s}}{1 - \rho_1^t}, \hat{\mathbf{r}} \leftarrow \frac{\mathbf{r}}{1 - \rho_2^t}$
 - Provedeme update vah: $\mathbf{w} \leftarrow \mathbf{w} - \epsilon \frac{\hat{\mathbf{s}}}{\delta + \sqrt{\hat{\mathbf{r}}}}$.

12. Vícevrstvé neuronové sítě - hluboké učení, regularizace

Problémy trénování hlubokých sítí

- Gradientní sestup mívá do směru nulového gradientu, což nemusí být globální minimum.

- Problém je, že u sítí s velkým počtem neuronů může snadno dojít k **přeučení**.
- Malý dataset se neuronka dokáže **naučit nazpaměť**.
- Používáme tedy různé druhy **regularizace**.

Metody regularizace - penalizace norem parametrů, předčasné zastavení, dropout, dávková normalizace

- **Penalizace norem parametrů**

- K účelové funkci $J(\mathbf{w})$ přidáme penalizační člen $\Omega(\mathbf{w})$.
- Nová účelová funkce je tedy

$$J'(\mathbf{w}) = J(\mathbf{w}) + \lambda \Omega(\mathbf{w}),$$

kde λ je hyperparametr kontrolující sílu regularizace.

- Penalizace se přidává po vrstvách.
- Při L_2 regularizaci pro i -tou vrstvu se přidá člen

$$\Omega(\mathbf{w}) = \|\mathbf{w}^{(i)}\|_2^2$$

- Při L_1 regularizaci pro i -tou vrstvu se přidá člen

$$\Omega(\mathbf{w}) = \|\mathbf{w}^{(i)}\|_1$$

- Regularizační efekt podobný jako u **Ridge** nebo **Lasso**.

- **Předčasné zastavení (early stopping)**

- Na konci každé epochy se spočte validační chyba a ukládá se ta nejlepší.
- Pokud se po l epochách nezmenší nejlepší validační chyba, ukončíme učení a vrátíme váhy odpovídající nejmenší validační chybě.

- **Dropout**

- Vynulování vybraných **vstupů do vrstvy**, na kterou je aplikován.
- Nulování jenom v trénovací fázi, **pro každou dávku nezávisle**.
- Každý vstup je nulován s pravděpodobností $p \in (0, 1)$.
- Nutí model soustředit se na různé aspekty a nesoustředit se pořád na to samé.
- Značně zvyšuje schopnost **generalizovat**.
- **Inverze dropout**
 - Nevynulované vstupy jsou ještě přeškálovány číslem $\frac{1}{1-p}$.

- **Dávková normalizace**

- Pro hlubší sítě jsou při gradientním sestupu **parametry v mělčích vrstvách ovlivňovány parametry z hlubších vrstev**.
- Proti tomuhle slouží dávková normalizace.
- Pro danou dávku dat výstupy z určené skryté vrstvy **standardizujeme** a poté (volitelně) **lineárně ztransformujeme**.
- Zbavíme se **komplikované interakce parametrů v hlubších vrstvách**.

13. Konvoluční neuronové sítě

Operace konvoluce a její vlastnosti

- **Definice konvoluce:**

Nechť f a g jsou dvě funkce reálné proměnné, absolutně integrovatelné na celém $\mathbb{R}$.

Konvoluce funkcí f a g je definována vztahem

$$(f * g)(t) = \int_{\tau} f(\tau)g(t - \tau)d\tau$$

pro každé $t \in \mathbb{R}$.

- **Vlastnosti:**

- Komutativita: $f * g = g * f$
- Vztah vůči derivaci: $(f * g)'(t) = (f * g')(t)$
- Ekvivariance vůči translaci: $(f * g)(a + t) = (f(x + a) * g)(t)$

- **Definice diskrétní konvoluce:**

Nechť $(x(i))_{i=-\infty}^{+\infty}$ a $(w(i))_{i=-\infty}^{+\infty}$ jsou dvě posloupnosti.

Diskrétní konvoluci posloupností x a w definujeme vztahem

$$(x * w)(t) = \sum_{i=-\infty}^{+\infty} x(i)w(t - i)$$

pro každé $t \in \mathbb{Z}$.

- **Vlastnosti:**

- Analogické s konvolucí pro funkce.

- **Konvoluce v neuronových sítích**

- Uvažujme 2D obrázek I o rozměrech $p_1 \times p_2$, kdy použijeme dvourozměrné jádro K o rozměrech $k_1 \times k_2$ a spočteme

$$S(i, j) = (I * K)(i, j) = \sum_m \sum_n I(m, n)K(i - m, j - n)$$

- Z komutativity

$$S(i, j) = (K * I)(i, j) = \sum_m \sum_n I(i - m, j - n)K(m, n)$$

- A se učíme složky K uděláme

$$\sum_m \sum_n I(i + m, i + n)K(-m, -n)$$

- Tomuto se říká **křížová korelace**

$$(I \star K)(i, j) = \sum_{m=1}^{k_1} \sum_{n=1}^{k_2} I(i + m - 1, j + n - 1)K(m, n)$$

kde $1 \leq i \leq p_1 - k_1 + 1$ a $1 \leq j \leq p_2 - k_2 + 1$.

Konvoluční vrstva

- Aplikujeme na d -rozměrné tenzory s c kanály.
- V **channel-first** notaci jednomu datovému bodu odpovídá tenzor $\mathbf{I}$ o rozměrech $(c_{\text{in}}, p_1, \dots, p_d)$.
- Jeden výstupní kanál konvoluční vrstvy s jádrem $\mathbf{K}$ o rozměrech $(k_1, \dots, k_d)$ má pro každý vstupní kanál jeden tenzor o těchto rozměrech a výsledné konvoluce (křížové korelace) se přes kanály sčítají.
- K tomuto se přičte ještě intercept b .
- Obecně na výstupu máme c_{out} kanálů a j -tý kanál výstupního tenzoru $\mathbf{O}$ o rozměrech $(c_{\text{out}}, p_1 - k_1 + 1, \dots, p_d - k_d + 1)$ je určen vztahem

$$\mathbf{O}_j = b_j + \sum_{l=1}^{c_{\text{in}}} \mathbf{I}_l \star \mathbf{K}_{j,l}.$$

- Složky jádra $\mathbf{K}$ i intercept b jsou parametry, které se trénují gradientním sestupem.

Porovnání s plně propojenými vrstvami - princip omezeného spojení a sdílení parametrů

- Konvoluční vrstva **snižuje počty parametrů** s klasickými plně propojenými vrstvami.
- Každý výstupní bod je propojen pouze s omezenou oblastí vstupu (**oblast vnímání**).
- Tomuto se říká **princip omezeného spojení**.
- **Sdílení parametrů**
 - **Konvoluční vrstvu** můžeme chápat jako **plně propojenou vrstvu s apriorně svázanými a lokálně omezenými parametry**.
 - **Apriorně svázané** = Apriorně (předem) se určí, že se pro různé části dat použijí naprosto stejné váhy. Vezmu malý filtr a tím projedu celý obrázek.
 - **Lokálně omezené** = Neuron se nekouká na všechna data, ale jen na malý výřez ve svém okolí.

Pooling, padding, stride

- **Pooling**
 - Konvoluční vrstva se typicky realizuje pomocí **konvoluce, aplikace nelineární aktivační funkce, sdružovací funkce**.
 - Funguje to jako taková **komprese**.
 - Vezmu čtvereček (třeba 2×2) a smrsknu ho do jednoho čísla.
 - Nejtypičtějším příkladem sdružovací funkce je **max pooling**, kde vezmu pixel s největší hodnotou.
 - Můžu použít i **average pooling** a podobné.
- **Padding**
 - Před výpočtem se rozšíří vstupní tenzor ve všech směrech o daný počet hodnot.

- Typicky doplňují nuly nebo hodnoty z okraje, či zrcadlení.
- **Stride**
 - Normálně posouváme indexy konvoluce nebo paddingu o jedna.
 - Tento **skok** se nazývá **stride** a můžeme skákat i o víc než jedna.
 - Ekvivalentní (ne však výpočetně) jako provedení původního výpočtu a vyházení části vypočítaných hodnot.

14. Strojové zpracování přirozeného jazyka (nebude zkoušeno)

Maticová reprezentace textů v bag of words modelu

Stop slova, lemmatizace, Tf-idf

15. Posilované učení

Formulace problému a základních pojmů posilovaného učení

- Uvažujeme systém, kde můžeme vykonávat **akce** a snažíme se dosáhnout nějakého **cíle**.
- Tomuto systému říkáme **agent**.
- **Posilované učení** je snaha naučit agenta se chovat tak, aby **maximalizovat** nějakou **odměnu**.
- Neříkáme, jaké přímo akce má agent vykonávat, ale agent musí zkoušením zjistit, co je nejvýhodnější.
- Snaží se maximalizovat **všechny budoucí odměny**.
- Agent se nachází v **prostředí**, které je v nějakém **stavu** a na základě toho se agent rozhodne, jakou akci vykoná.
- Typicky obdrží po vykonání akce **okamžitou odměnu**.
- **Příklady:**
 - Piškvorky
 - Šachy
 - Go
 - apod.

Explorace versus exploitace

Agent se neustále snaží balancovat mezi dvěma přístupy.

- **Explorace**
 - Agent zkouší různé možnosti a snaží se zjistit, jaké akce jsou pro něj nejvýhodnější.
- **Exploitace**
 - Agent na základě znalostí z explorace opakuje tu akci, která mu přináší největší

odměnu.

- **Problém explorace vs exploitace:**

- Když agent bude trávit příliš mnoho času v **explorační** fázi, bude **zbytečně trávit čas neoptimálními kroky a přijde o možnou odměnu**.
- Když ale zase příliš brzo vkročíme do fáze **exploitace**, můžeme minout lepší řešení a *exploitování* tohoto **horšího řešení nepovede k maximalizaci odměny**.

***k*-ruký bandita a základní algoritmy jeho řešení**

- ***k*-ruký bandita:** Máme *k* dispozici několik akcí, každá s jinou distribucí odměn.
- A_t : **Akce** zvolená v čase t .
- R_t : **Odměna** obdržena v čase t po vykonání akce.
- $q_*(a)$: Očekávaná odměna při zvolené akci a , nazýváme **hodnota akce a** . Definujeme jako $q_*(a) = \mathbb{E}[R_t | A_t = a]$.
- Funkce $q_*(a) : \mathcal{A} \rightarrow \mathbb{R}$ se nazývá **hodnotová funkce**.
- Kdybychom znali $q_*(a)$ všech akcí, tak optimální strategie bude vždy volit $\arg \max_a q_*(a)$.
- To ale neznáme a tak v průběhu vytváříme **odhad**.
- $Q_t(a)$: **Odhad hodnoty akce a** v čase t .
- Nejpřirozenější způsob sestrojení odhadu je použití výběrového průměru odměn

$$Q_t(a) = \frac{\sum_{\tau=1}^{t-1} R_\tau \mathbb{1}_{A_\tau=a}}{\sum_{\tau=1}^{t-1} \mathbb{1}_{A_\tau=a}}.$$

- Ze zákona velkých čísel tento odhad konverguje ke střední hodnotě $q_*(a)$.

- **Algoritmy**

- **greedy search**

- Vybereme akci A_t jako

$$A_t = \arg \max_a Q_t(a).$$

- Pokud je Q_t nejvyšší pro několik akcí, vybereme náhodně.

- **Nevýhody:**

- Nevhodné, když naše odhady $Q_t(a)$ nejsou moc dobré.
 - Pokud na začátku dostaneme špatnou odměnu, už se z toho nevyhrabeme.
 - Nevhodné pro nestacionární systém (mění se v čase).

- **ϵ -greedy search**

- Inicializujeme pro všechna a : $Q(a) = 0$ a $N(a) = 0$.

- Zvolíme parametr ϵ .

- **Opakujeme:**

- $A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{s pst } 1 - \epsilon \\ \text{random action} & \text{s pst } \epsilon \end{cases}$
 - $R \leftarrow \text{bandit}(A)$

- $N(A) \leftarrow N(A) + 1$
- $Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$
- **Výhody:**
 - V limitě najde optimum.
- **Nevýhody:**
 - Po nalezení optima stále bude zkoušet s pst ϵ nevýhodné akce.
 - ϵ je parametr, který musíme ladit.
- **Optimistický greedy search**
 - Zavedeme apriorní znalost systému.
 - Dosáhneme tak zlepšení explorační.
 - K tomu stačí nastavit $Q(a)$ na stejné, vysoké hodnoty.
 - **Výhody:**
 - Silně a systematicky podporuje exploraci na začátku.
 - Mnohokrát konverguje rychleji než ϵ -greedy.
 - **Nevýhody:**
 - V nestacionárním systému selhává, protože prvotní optimismus vyprchá a pak je to čistý greedy search.
 - Vyžaduje expertní znalost systému (maximální možná odměna) k nastavení počáteční hodnoty.
- **UCB (Upper Confidence Bound):**
 - Zavádí **systematickou exploraci** (místo náhodné pomocí ϵ).
 - A_t volíme jako:

$$A_t = \arg \max_a \left[Q_t(a) + c \cdot \sqrt{\frac{\ln t}{N_t(a)}} \right],$$

kde c je parametr kontrolující míru explorační.

- Kombinuje se snaha **vybrat aktuálně nejlepší stav** se snahou **redukovat nejistotu**.
- **Výhody:**
 - Cíleně vybírá akce, které jsou buď velmi slibná nebo málo prozkoumaná.
- **Nevýhody:**
 - Problémy v nestacionárním případě.
- *Markovský rozhodovací problém: Nezohledňují historii a řídí se pouze současným stavem a prostředím.*